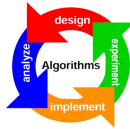


Estrategias algorítmicas

José A. Rodríguez Melquiades

17 de mayo de 2026

Estrategia algorítmica: Fuerza bruta



1 Algoritmos por la fuerza bruta

- Problemas solucionados con fuerza bruta
- Problema de la explosión

2 Coincidencia de cadenas

Algoritmos por la fuerza bruta

- **Algoritmos de fuerza bruta**¹: Técnicas de búsqueda exhaustiva que, para hallar la solución óptima, evalúan sistemáticamente todas las posibles soluciones.
Conceptualmente son simples, pero pueden ser computacionalmente costosos debido a la exploración completa del espacio de búsqueda.

¹Brute force

Algoritmos por la fuerza bruta

- **Algoritmos de fuerza bruta**¹: Técnicas de búsqueda exhaustiva que, para hallar la solución óptima, evalúan sistemáticamente todas las posibles soluciones.
Conceptualmente son simples, pero pueden ser computacionalmente costosos debido a la exploración completa del espacio de búsqueda.
- **Definición**: Algoritmo por fuerza bruta es aquel que examina todas las soluciones posibles y elige la mejor según algún criterio predefinido.

¹Brute force

Algoritmos por la fuerza bruta

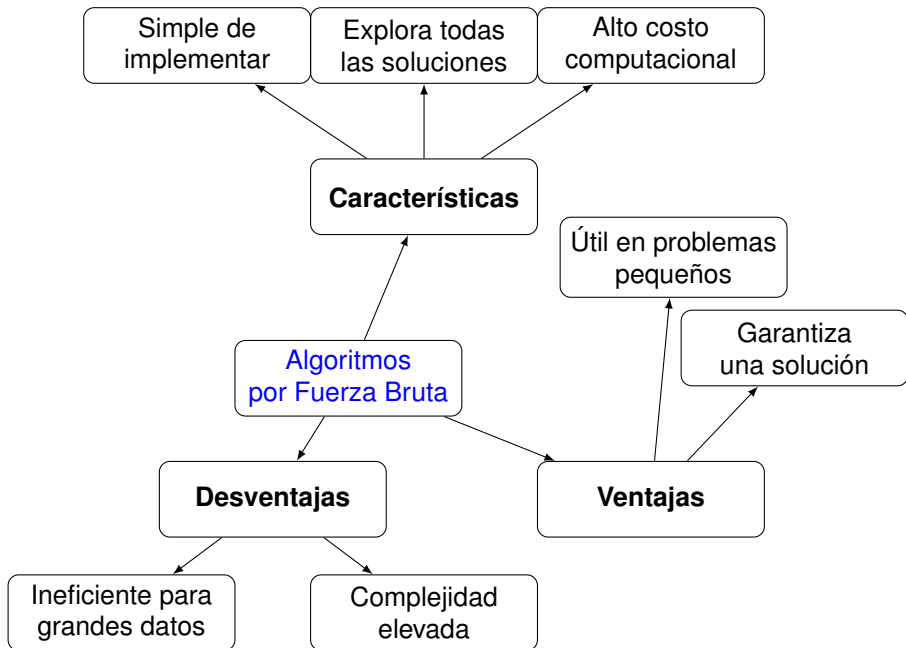
- **Algoritmos de fuerza bruta**¹: Técnicas de búsqueda exhaustiva que, para hallar la solución óptima, evalúan sistemáticamente todas las posibles soluciones.
Conceptualmente son simples, pero pueden ser computacionalmente costosos debido a la exploración completa del espacio de búsqueda.
- **Definición**: Algoritmo por fuerza bruta es aquel que examina todas las soluciones posibles y elige la mejor según algún criterio predefinido.
- **Observación**:

¹Brute force

Algoritmos por la fuerza bruta

- **Algoritmos de fuerza bruta¹**: Técnicas de búsqueda exhaustiva que, para hallar la solución óptima, evalúan sistemáticamente todas las posibles soluciones.
Conceptualmente son simples, pero pueden ser computacionalmente costosos debido a la exploración completa del espacio de búsqueda.
- **Definición**: Algoritmo por fuerza bruta es aquel que examina todas las soluciones posibles y elige la mejor según algún criterio predefinido.
- **Observación**:
 - ▶ No utiliza heurísticas ni estrategias sofisticadas para reducir el espacio de búsqueda.

¹Brute force



● Seudo código:

```
1: Definir el Espacio de Búsqueda (EB)
2: for cada solución candidata en el EB do
3:     Verificar si solución candidata cumple con condiciones requeridas
4:     if Cumple then
5:         if se busca una sola solución válida then
6:             Retornar la solución
7:         end if
8:         if se busca la mejor solución then
9:             Evaluar la calidad de la solución
10:            if es mejor que la mejor encontrada hasta ahora then
11:                Guardar como nueva mejor solución
12:            end if
13:        end if
14:    end if
15: end for
16: if se encontró al menos una solución válida then
17:     Retornar la mejor o las válidas encontradas
18: else
19:     Informar que no hay solución válida
20: end if
```

Problemas solucionados con fuerza bruta

- Problemas combinatorios y de búsqueda:

Problemas solucionados con fuerza bruta

- Problemas combinatorios y de búsqueda:
 - ▶ Problema del agente viajero(TSP): Busca todas las posibles combinaciones o permutaciones de recorridos y elige la mas corta.

Problemas solucionados con fuerza bruta

- Problemas combinatorios y de búsqueda:

- ▶ Problema del agente viajero(TSP): Busca todas las posibles combinaciones o permutaciones de recorridos y elige la mas corta.
- ▶ Problema de la mochila (Knapsack Problem): Evalua todas las combinaciones de objetos, sin exceder la capacidad de la mochila, para maximizar el valor.

Problemas solucionados con fuerza bruta

- Problemas combinatorios y de búsqueda:

- ▶ Problema del agente viajero(TSP): Busca todas las posibles combinaciones o permutaciones de recorridos y elige la mas corta.
- ▶ Problema de la mochila (Knapsack Problem): Evalua todas las combinaciones de objetos, sin exceder la capacidad de la mochila, para maximizar el valor.
- ▶ Satisfacción booleana (SAT): Combinaciones posibles de valores de verdad para variables lógicas.

- Problemas búsqueda:

- Problemas búsqueda:

- ▶ Búsqueda de una subcadena en una cadena (String matching): Comparar cada posición del texto(T) con el patrón (P).

- Problemas búsqueda:

- ▶ Búsqueda de una subcadena en una cadena (String matching): Comparar cada posición del texto(T) con el patrón (P).
- ▶ Búsqueda de claves en ataques de fuerza bruta a contraseñas: Probar todas las combinaciones posibles de caracteres hasta hallar la contraseña correcta.

- Problemas búsqueda:

- ▶ Búsqueda de una subcadena en una cadena (String matching): Comparar cada posición del texto(T) con el patrón (P).
- ▶ Búsqueda de claves en ataques de fuerza bruta a contraseñas: Probar todas las combinaciones posibles de caracteres hasta hallar la contraseña correcta.

- Problemas geométricos:

- Problemas búsqueda:

- ▶ Búsqueda de una subcadena en una cadena (String matching): Comparar cada posición del texto (T) con el patrón (P).
- ▶ Búsqueda de claves en ataques de fuerza bruta a contraseñas: Probar todas las combinaciones posibles de caracteres hasta hallar la contraseña correcta.

- Problemas geométricos:

- ▶ Par de puntos mas cercanos: Dado un plano, comparar todas las distancias entre pares de puntos.

- Problemas búsqueda:

- ▶ Búsqueda de una subcadena en una cadena (String matching): Comparar cada posición del texto (T) con el patrón (P).
- ▶ Búsqueda de claves en ataques de fuerza bruta a contraseñas: Probar todas las combinaciones posibles de caracteres hasta hallar la contraseña correcta.

- Problemas geométricos:

- ▶ Par de puntos mas cercanos: Dado un plano, comparar todas las distancias entre pares de puntos.
- ▶ Cerradura convexa (Convex Hull): Probar todas las combinaciones de puntos para formar la envolvente.

- Problemas de asignación y permutaciones:

- Problemas de asignación y permutaciones:
 - ▶ Permutación de n elementos: Generar todas las permutaciones posibles y evaluar una condición específica.

- Problemas de asignación y permutaciones:

- ▶ Permutación de n elementos: Generar todas las permutaciones posibles y evaluar una condición específica.
- ▶ Problema de asignación (Assignment Problem): Explorar todas las asignaciones posibles entre tareas y agentes o personas.

- Problemas de asignación y permutaciones:

- ▶ Permutación de n elementos: Generar todas las permutaciones posibles y evaluar una condición específica.
- ▶ Problema de asignación (Assignment Problem): Explorar todas las asignaciones posibles entre tareas y agentes o personas.
- ▶ N-reinas: Colocar N reinas en un tablero sin que se ataquen entre ellas, para ello probar todas las combinaciones posibles.

- Problemas de asignación y permutaciones:

- ▶ Permutación de n elementos: Generar todas las permutaciones posibles y evaluar una condición específica.
- ▶ Problema de asignación (Assignment Problem): Explorar todas las asignaciones posibles entre tareas y agentes o personas.
- ▶ N-reinas: Colocar N reinas en un tablero sin que se ataquen entre ellas, para ello probar todas las combinaciones posibles.

- Problemas de optimización:

- Problemas de asignación y permutaciones:

- ▶ Permutación de n elementos: Generar todas las permutaciones posibles y evaluar una condición específica.
- ▶ Problema de asignación (Assignment Problem): Explorar todas las asignaciones posibles entre tareas y agentes o personas.
- ▶ N-reinas: Colocar N reinas en un tablero sin que se ataquen entre ellas, para ello probar todas las combinaciones posibles.

- Problemas de optimización:

- ▶ Problema de corte de varillas (Rod Cutting): Probar todas las formas posibles de cortar una varilla para maximizar el beneficio.

- Problemas de asignación y permutaciones:

- ▶ Permutación de n elementos: Generar todas las permutaciones posibles y evaluar una condición específica.
- ▶ Problema de asignación (Assignment Problem): Explorar todas las asignaciones posibles entre tareas y agentes o personas.
- ▶ N-reinas: Colocar N reinas en un tablero sin que se ataquen entre ellas, para ello probar todas las combinaciones posibles.

- Problemas de optimización:

- ▶ Problema de corte de varillas (Rod Cutting): Probar todas las formas posibles de cortar una varilla para maximizar el beneficio.
- ▶ Subset Sum Problem: Probar todos los subconjuntos posibles para ver si alguno de ellos suma un valor objetivo buscado.

Problema de la explosión

- **Buscar mediante fuerza bruta:** La búsqueda combinatoria y búsqueda exhaustiva, es considerado una técnica trivial (sencilla) pero muy usada, el cual consiste en hacer una enumeración de todos los posibles candidatos a ser solución del problema dado.

Problema de la explosión

- **Buscar mediante fuerza bruta:** La búsqueda combinatoria y búsqueda exhaustiva, es considerado una técnica trivial (sencilla) pero muy usada, el cual consiste en hacer una enumeración de todos los posibles candidatos a ser solución del problema dado.
- **Observación:**
En la mayoría de los problemas reales la cantidad de **”soluciones candidatas generadas es elevado”**, por ello se le denomina **”el problema de la explosión”**.

- Ejemplo:

- Ejemplo:

- **Problema de las N reinas:** Colocar N reinas del juego de ajedrez en un tablero de tamaño $N \times N$, tal que dos reinas no se ataquen entre sí, es decir, no deben haber dos reinas que compartan la misma fila, la misma columna o la diagonal.

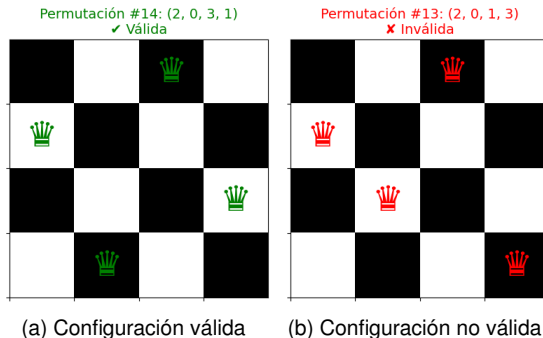


Figura 1: Problema de 4-reinas: $4!$ permutaciones

- Ejemplo

²Salesman problem y vehicle routing problem

● Ejemplo

- **Problemas de ruteo:**² Hallar la mejor solución puede implicar encontrar todas las soluciones o si se conoce un valor para la mejor solución, el proceso puede detenerse cuando encuentra la respuesta.

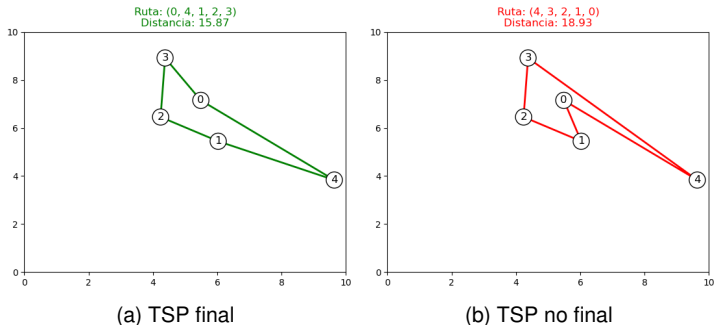


Figura 2: TSP: $5! = 120$ rutas

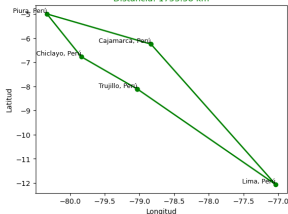
²Salesman problem y vehicle routing problem

- Ejemplo (cont)

● Ejemplo (cont)

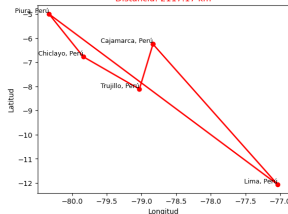
► Problemas de ruteo:(cont.)

Ruta: ['Trujillo, Perú', 'Chiclayo, Perú', 'Piura, Perú', 'Cajamarca, Perú', 'Lima, Perú']
Distancia: 1753.58 km



(a) TSP final

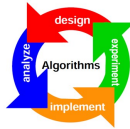
Ruta: ['Cajamarca, Perú', 'Lima, Perú', 'Piura, Perú', 'Chiclayo, Perú', 'Trujillo, Perú']
Distancia: 2117.17 km



(b) TSP no final

Figura 3: TSP: $5! = 120$ rutas entre ciudades del Perú

Estrategia algorítmica: Fuerza bruta



1 Algoritmos por la fuerza bruta

- Problemas solucionados con fuerza bruta
- Problema de la explosión

2 Coincidencia de cadenas

Coincidencia de cadenas

- Dada una cadena de n caracteres llamada **Texto** y otra de m caracteres ($m \leq n$) llamada **Patrón**. **Hallar una subcadena** del texto que coincida con el patrón.

Coincidencia de cadenas

- Dada una cadena de n caracteres llamada **Texto** y otra de m caracteres ($m \leq n$) llamada **Patrón**. Hallar una subcadena del texto que coincida con el patrón.
- Se desea encontrar i , el índice del carácter más a la izquierda de la primera subcadena coincidente en el texto, tal que

$$t_i = p_0, p_1, \dots, t_{i+j} = p_j, p_{j+1}, \dots, t_{i+m-1} = p_{m-1}$$

Coincidencia de cadenas

- Dada una cadena de n caracteres llamada **T**exto y otra de m caracteres ($m \leq n$) llamada **P**atrón. Hallar una subcadena del texto que coincida con el patrón.
- Se desea encontrar i , el índice del carácter más a la izquierda de la primera subcadena coincidente en el texto, tal que

$$t_i = p_0, p_1, \dots, t_{i+j} = p_j, p_{j+1}, \dots, t_{i+m-1} = p_{m-1}$$

- Alinear **P**, con sus primeros m caracteres, con el **T** y hacerlos coincidir entre pares de caracteres de izquierda a derecha, hasta que todos los pares de caracteres coincidan. Luego el algoritmo se detiene o encuentra un par que no coincide.

$t_0 \dots t_i \dots t_{i+j} \dots t_{i+m-1} \dots t_{n-1}$



$p_0 \dots p_j \dots p_{m-1}$

- Seudocódigo:

```
1: Input: Texto  $T[0 \dots n - 1]$  de  $n$  caracteres.  
2:      Patron  $P[0 \dots m - 1]$  de  $m$  caracteres.  
3: Output: El índice del primer carácter del texto que comienza en  
      subcadena coincidente o  $-1$  si la búsqueda no tiene éxito.  
  
4: for  $i := 0$  to  $n - m$  do  
5:    $j := 0$   
6:   while  $j < m$  and  $P[j] = T[i + j]$  do  
7:      $j := j + 1$   
8:   end while  
9:   if  $j = m$  then  
10:    Return  $i$   
11:  end if  
12:  Return  $-1$   
13: end for
```

- Ejemplo:

- Ejemplo:

- ▶ Texto: EjemploCorreosElectrónicosdeEstudiantesUNTCorreosElectrónicos

- Ejemplo:

- ▶ Texto: EjemploCorreosElectrónicosdeEstudiantesUNTCorreosElectrónicos
- ▶ Patrón: CorreosElectrónicos

- ▶

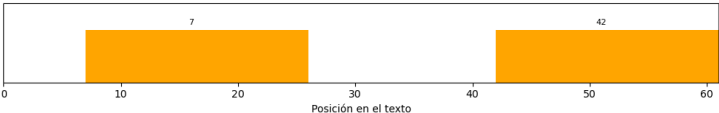


Figura 4: Búsqueda de coincidencias en un texto.

- Aplicaciones:

- **Aplicaciones:**

- ▶ **Detección de plagio:** Se utiliza la coincidencia de cadenas para comprobar si un documento es una copia, para ello se divide el contenido en fragmentos más pequeños de texto, como palabras u oraciones, y los compara con otros documentos. Si encuentra mucha similitud, el documento esta plagiado.

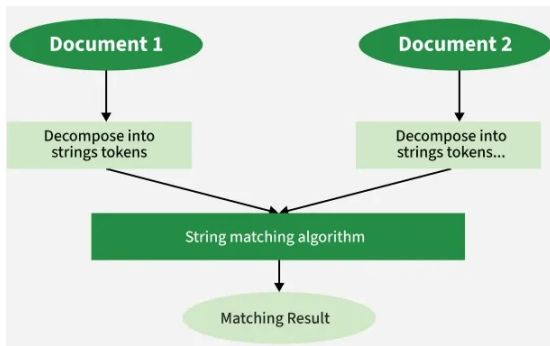


Figura 5: Compara coincidencias entre textos.

- Aplicaciones:(cont.)

- **Aplicaciones:(cont.)**

- ▶ **Bioinformática y secuenciación de ADN:** El ADN es una cadena compuesta por A, T, G y C, de este modo la comparación ayuda a buscar patrones específicos en el ADN. Esto puede ayudar a identificar enfermedades, mutaciones o la compatibilidad de genes en diferentes organismos.

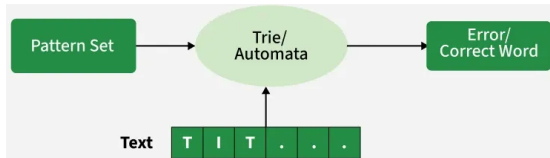


Figura 6: La comparación busca patrones específicos en el ADN.

- Aplicaciones:(cont.)

- Aplicaciones:(cont.)

- ▶ **Corrector ortográfico:** Se almacena en la memoria un diccionario de palabras correctas los cuales serán comparados con el patrón mediante Trie o Autómata.

- Aplicaciones:(cont.)

- ▶ **Corrector ortográfico:** Se almacena en la memoria un diccionario de palabras correctas los cuales serán comparados con el patrón mediante Trie o Autómata.
- ▶ **Filtros de spam:** Los filtros de spam utilizan la coincidencia de cadenas para comprobar si un correo electrónico contiene palabras clave como "ganar dinero" o "haga clic aquí". Si se encuentran estas palabras, el correo electrónico se marca como spam.

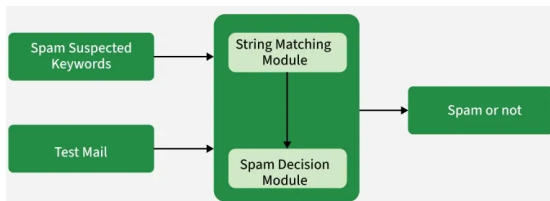


Figura 7: Spam: correos electrónicos con contenido no deseado.

- Aplicaciones:(cont.)

- **Aplicaciones:(cont.)**

- ▶ **Motores de búsqueda y grandes bases de datos:** Compara las palabras escritas con el contenido almacenado en sitios web o bases de datos.

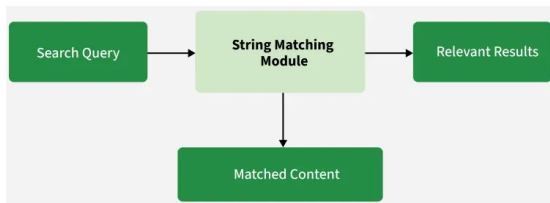


Figura 8: Comparación de palabras.

- Aplicaciones:(cont.)

- Aplicaciones:(cont.)

- **Sistema de detección de datos maliciosos:** Usado para proteger las computadoras de *hackers* y virus, por ello la coincidencia de cadenas busca patrones peligrosos conocidos (como código de virus). Si se encuentra una coincidencia, el sistema activa una alarma para bloquear la amenaza.

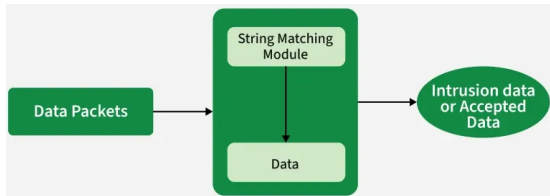


Figura 9: Sistemas monitorean todos los datos que entran en la red.